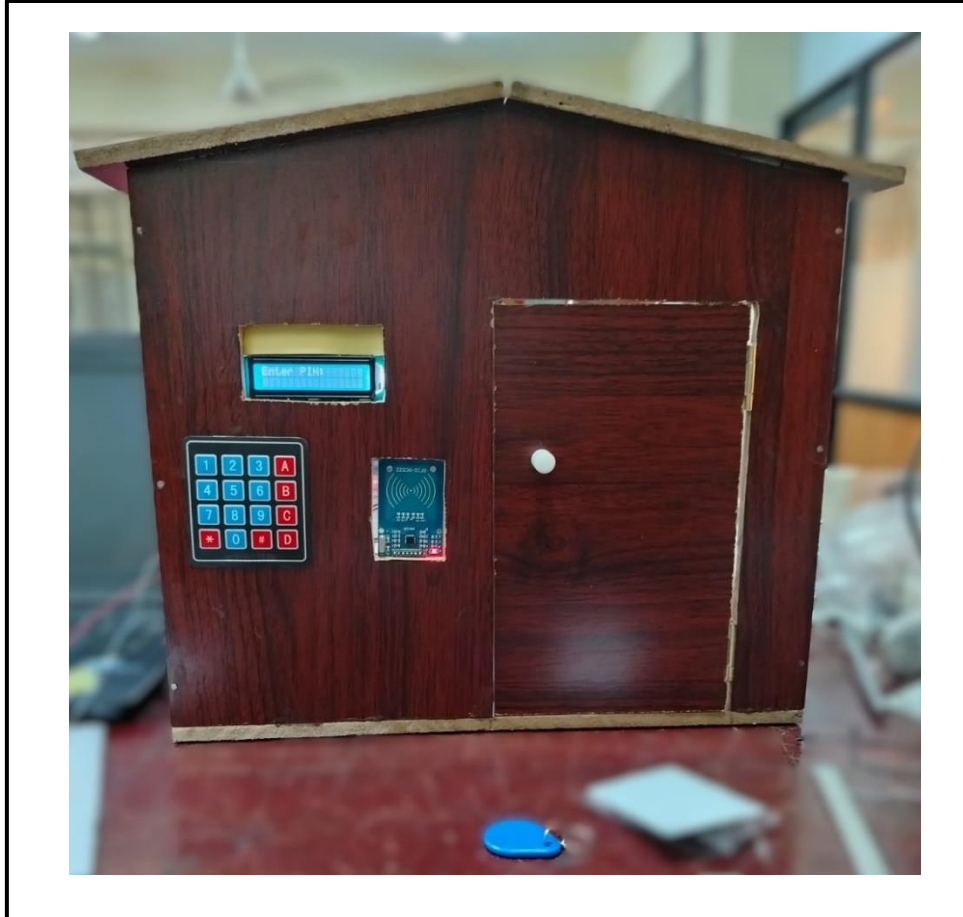


Smart Vault Door Lock



Report on the Design and Implementation of a Dual-factor Authentication Vault Door Lock

PH3032- Embedded Laboratory System

NAME: H.S.D. Hewanadugala

REG. NO: 2022S19405

INDEX NO: S16802

DATE: 14/09/2025

Table of Contents

1.0 Abstraction	4
2.0 Introduction	5
3.0 Methodology: Hardware Components and Circuit Design	6
3.1 Microcontroller: ATmega328p Pinout Diagram	6
3.2 PCB Diagram	7
3.3 I ² C Communication Protocol (SDA - PC4, SCL - PC5)	9
3.4 4x4 Matrix Keypad (Via PCF8574)	9
3.5 RFID Module (MFRC522)	11
3.6 I ² C LCD Display	13
3.7 Solenoid Lock Actuator (PD4)	14
3.8 LED and Buzzer Feedback System (PC0, PC1, PC2)	16
3.9 Exit Button (PB0)	16
3.10 Reset Button (PD7)	16
3.11 SIM900A GSM Module	18
4.0 Results and Analysis	20
5.0 Discussion	22
6.0 Conclusion	23
7.0 References	24
8.0 Appendix	25

Table of Figures

Figure 1: ATmega328p Microcontroller Pinout Diagram.....	6
Figure 2: Front side of the PCB	7
Figure 3: Backside of the PCB.....	7
Figure 4: Fully designed PCB with the components.....	8
Figure 5: LCD with I2C PCF8574	9
Figure 6: PCF8574 HW400 Keypad used I2C.....	9
Figure 7: 4x4 matrix keypad with connected PCF8574 I2C.....	9
Figure 8: MFRC522 RFID module with blue tag and card	11
Figure 9: Connected prototype with keypad, LCD and RFID.....	11
Figure 10: Backside of the connected prototype with the solenoid lock	14
Figure 11: 12V Solenoid Lock	15
Figure 12: GSM Module SIM900A.....	16

1.0 Abstraction

This report details the design, implementation and functionality of a secure vault door lock system utilizing dual-factor authentication. The system enhances security by requiring two independent verification methods: a 6-digit alphanumeric code entered using a 4x4 matrix keypad and the scanning of an authorized RFID card. The system is built around an ATmega328p microcontroller and incorporates robust error handling mechanisms including a limited number of attempts for each stage, visual and audible feedback via LEDs and a buzzer, and a real-time status display on an LCD. For enhanced security monitoring, the system integrates a GSM communication module that automatically sends instant alert messages to authorized personnel when multiple unauthorized access attempts are detected, ensuring immediate response to potential security breaches. The project successfully demonstrates a practical application of embedded systems in security, integrating multiple communication protocols such as I²C, SPI, and GSM connectivity for comprehensive protection.

2.0 Introduction

In today's world, safeguarding valuable items, information and restricted areas is more important than ever. Conventional single step security methods like typing a numeric code or swiping an identification card are no longer sufficient on their own. They can be easily compromised by tactics such as password cracking, observational techniques or physical theft or credentials. This clear vulnerability highlights the urgent need for stronger access control systems that rely on multiple separate forms of identity confirmation and incorporate real-time remote alert capabilities.

This project addresses this pressing security challenge through the design and implementation of a **dual authentication vault door lock system** with integrated remote monitoring. The core objective is to engineer a reliable, hardware-based security solution that mandates two distinct factors of identification that **something you know (a secret PIN) and something you have (an authorized RFID card) before granting physical access**. This approach, known as multi-factor authentication, significantly raises the security threshold by ensuring that a compromise of one factor does not lead to a full system breach. Furthermore, the system incorporates a GSM-based alert mechanism that immediately notifies authorized personnel via SMS of any security breaches or repeated unauthorized access attempts, enabling rapid response to potential threats.

This system is built around an ATmega328p microcontroller, serving as the central processing unit that orchestrates a suite of peripheral devices. These include a 4x4 matrix keypad for PIN entry, an MFRC522 RFID reader for card verification, a 16x2 LCD for user feedback, a solenoid lock as the physical actuating mechanism, and a SIM900A GSM module for remote communication and emergency alerts. To enhance user interaction and system security, visual and audible indicators (LEDs and a buzzer) provide immediate feedback on the authentication status, while the GSM module ensures off-site monitoring capability.

The system incorporates a strict attempt limiting mechanism where exceeding three failed attempts in either authentication stage results in a complete system lockout and triggers an immediate SMS alert to pre-authorized security personnel. This functionality, combined with clear on-screen messaging and remote notification capabilities, ensures the system is not only secure but also intuitive for authorized users, decisively restrictive towards unauthorized entities, and responsive to potential security breaches. This report will comprehensively detail the system's architecture including its hardware components and circuit design, the software logic and algorithms governing its operation, the testing procedures undertaken to validate its functionality and a discussion of the results and potential future enhancements.

3.0 Methodology: Hardware Components and Circuit Design

3.1 Microcontroller: ATmega328p Pinout Diagram

ATmega328p: The core of the system running at 16MHZ. It executes the control logic, manages communication protocols (I₂C and SPI) and processes all inputs and outputs.

The ATmega328p serves as the central processing unit of the system. Its firmware is written in C and implements the main control logic, managing the sequence of authentication, handling input from the keypad and RFID reader, controlling output devices and enforcing the security policy (attempt limits). The microcontroller's internal resources such as timers for delays and I/O ports for peripheral control are efficiently utilized.

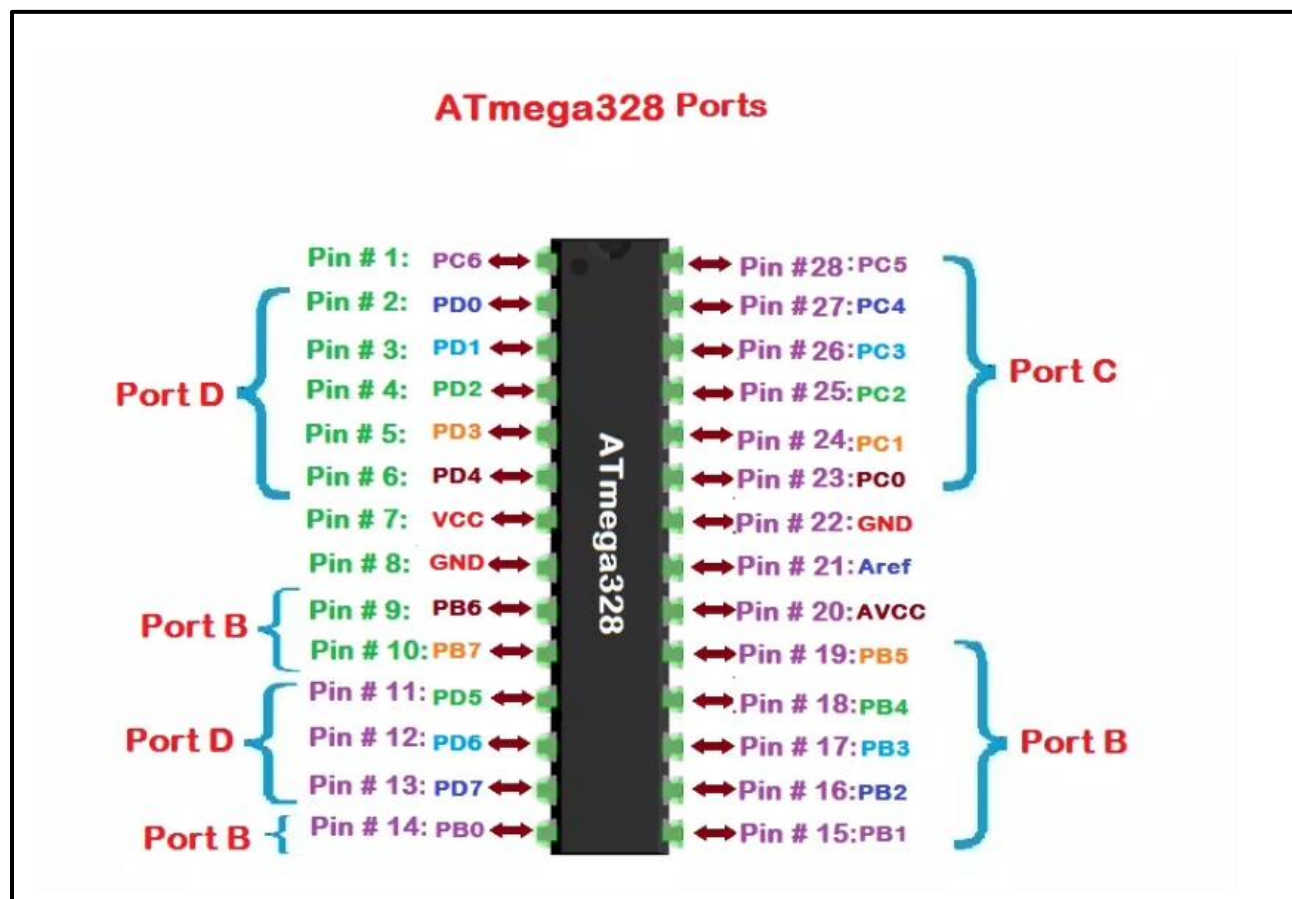


Figure 1: ATmega328p Microcontroller Pinout Diagram

3.2 PCB Diagram

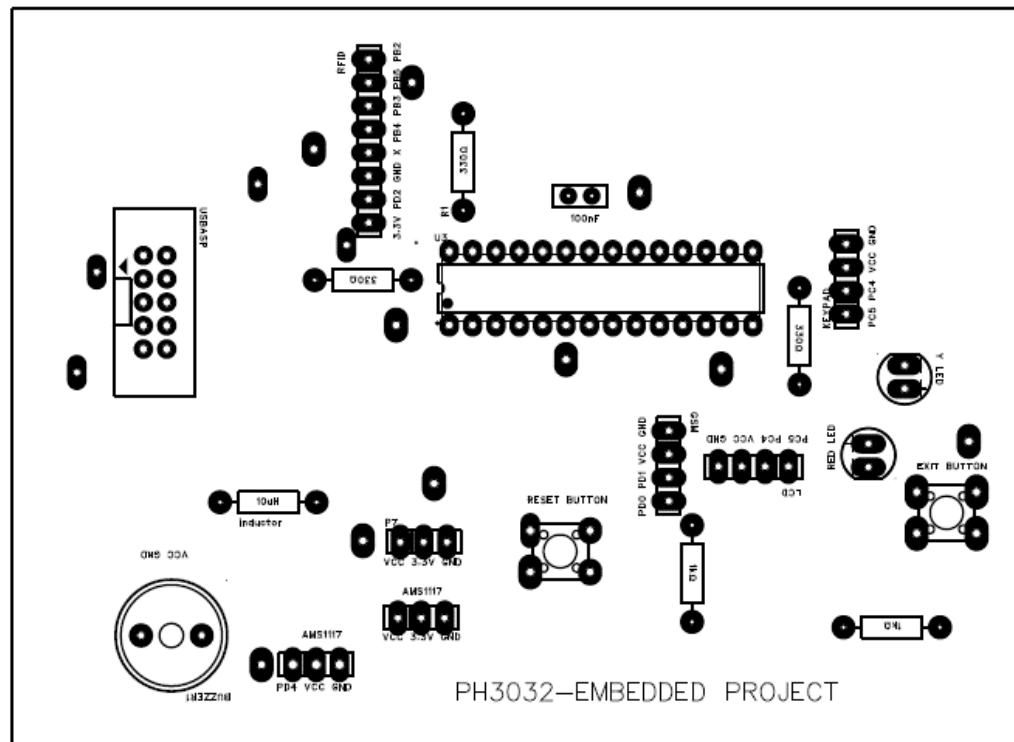


Figure 2: Front side of the PCB

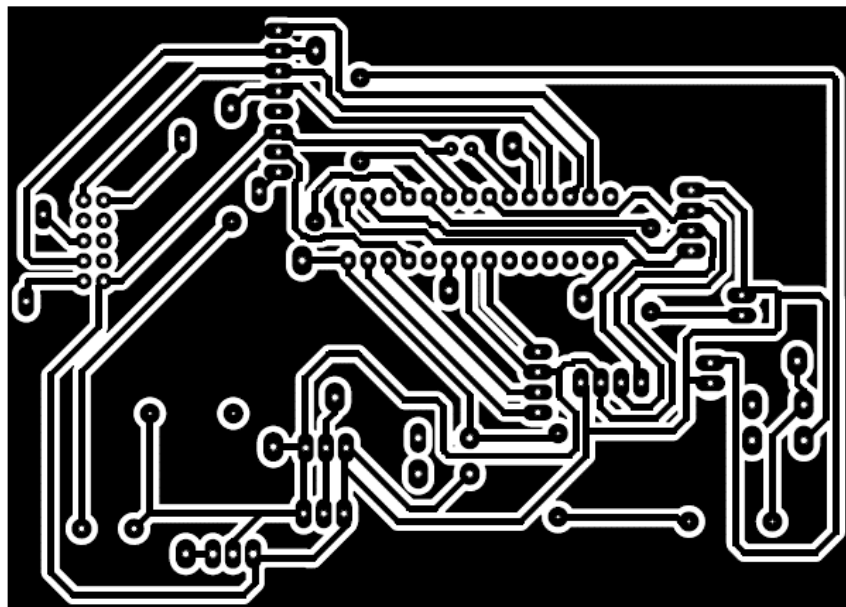


Figure 3: Backside of the PCB

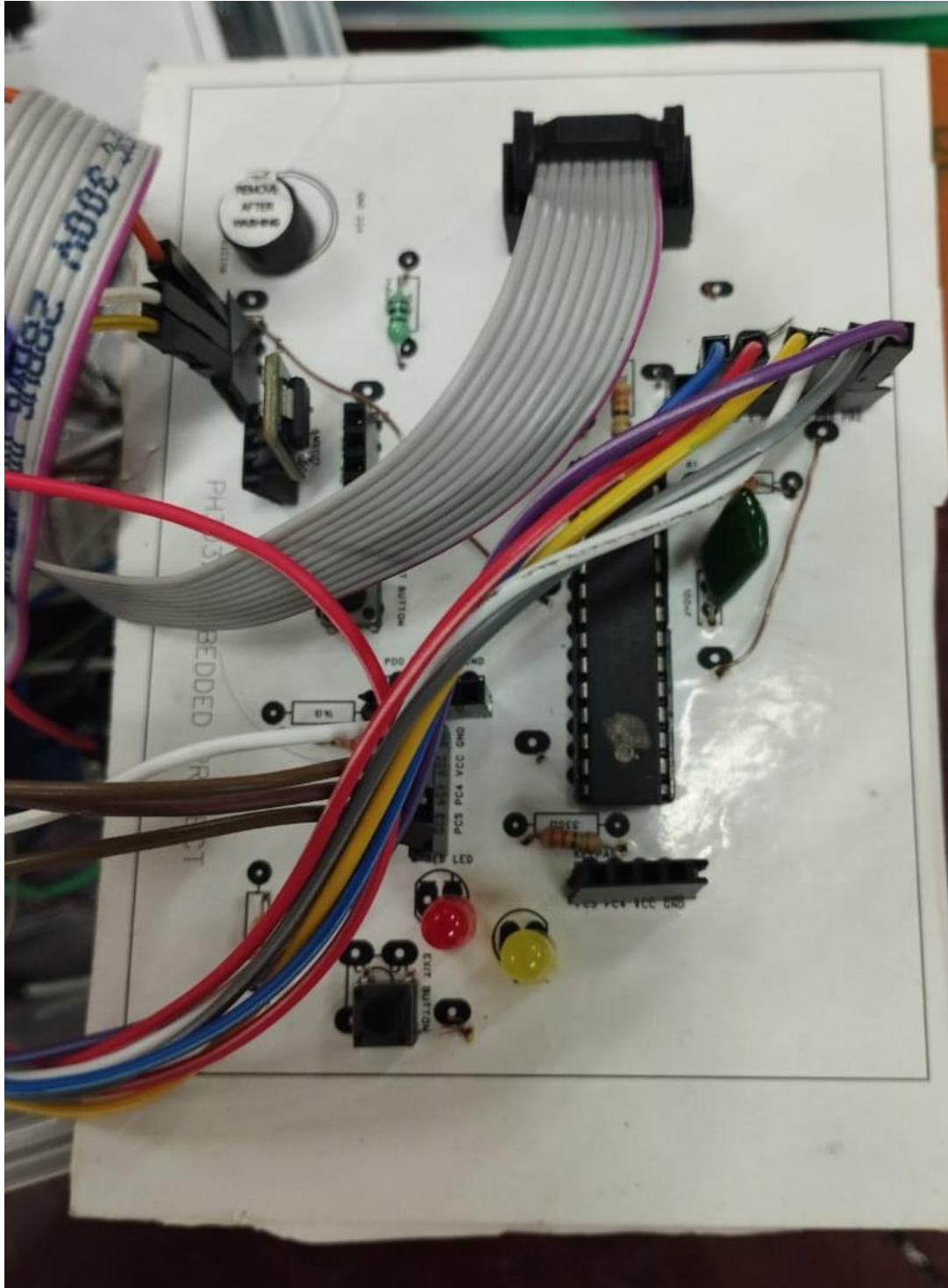


Figure 4: Fully designed PCB with the components

3.3 I²C Communication Protocol (SDA - PC4, SCL - PC5)

The I²C (TWI) protocol is used to communicate with the PCF8674 based keypad and the LCD display, minimizing the number of I/O pins required. The software includes functions for I²C initialization (I2C_init), start condition (I2C_start), data writing (I2C_write) and stop condition (I2C_stop).



Figure 5: LCD with I2C PCF8574



Figure 6: PCF8574 HW400 Keypad used I2C

3.4 4x4 Matrix Keypad (Via PCF8574)

Interface: I²C (TWI). The keypad is connected using a PCF8574 HW400 I²C port expander (address 0x20) to save precious I/O pins on the microcontroller. The keypad_getkey() function scans the keypad rows and columns by writing to and reading from the PCF8574.

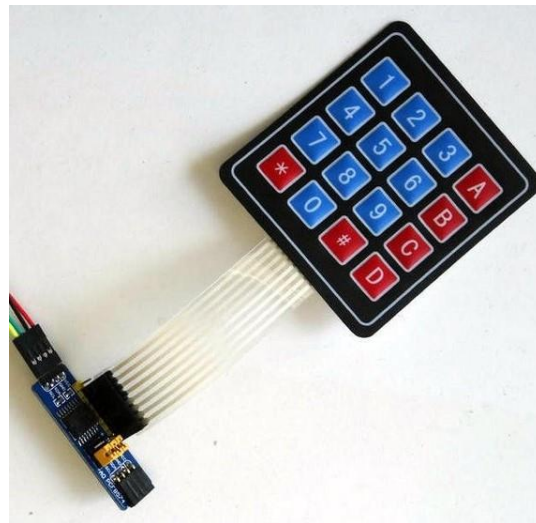


Figure 7: 4x4 matrix keypad with connected PCF8574 I2C

The 4x4 matrix keypad serves as the first authentication allowing the user to input a 6-digit alphanumeric PIN code. To conserve the microcontroller's valuable, I/O pins, the keypad is interfaced using a PCF8574 I²C port expander.

- a. The keypad is organized in a 4-row by 4-column matrix. The PCF8574 (with I²C address 0x20) is used to drive the rows low and read the state of the columns effectively managing the 8 required signals (4 rows + 4 columns) through the two wire I²C bus.
- b. The keypad_getkey() function scans the keypad by iterating through each row. It wires a byte to the PCF8574 to set the current row low and then reads a byte to check the state of all columns.
- c. A key press is detected when a specific column reads as low. The function incorporates software debouncing and key release detection to ensure accurate and reliable input registration preventing false triggers from signal noise.

```
const char keymap[4][4] = {  
    {'1', '2', '3', 'A'}, // Row 0  
    {'4', '5', '6', 'B'}, // Row 1  
    {'7', '8', '9', 'C'}, // Row 2  
    {'*', '0', '#', 'D'}  // Row 3
```

- d. keypad_getkey(): Scans the entire keypad matrix via the PCF8574. It returns the character value of the pressed key once it has been debounced and released.
- e. PIN Entry Logic: The main program loop calls this function and builds the user's input string. The “#” key is used to submit the PIN for verification while the “*” key is used to clear the current input.

The implementation provides a secure and efficient method for user input, forming the critical “Something you know” element of the dual authentication system.

3.5 RFID Module (MFRC522)

The MFRC522 RFID reader communicates with the microcontroller via the SPI protocol. The software includes low level functions to interact with the reader's registers (MFRC522_WriteRegister, MFRC522_ReadRegister). The checkRFID() function requests a card and reads its Unique Identifier (UID) comparing it against a hard coded authorized UID (0xE3, 0x38, 0xA1, 0x2C) and a specific wrong UID to determine access right.

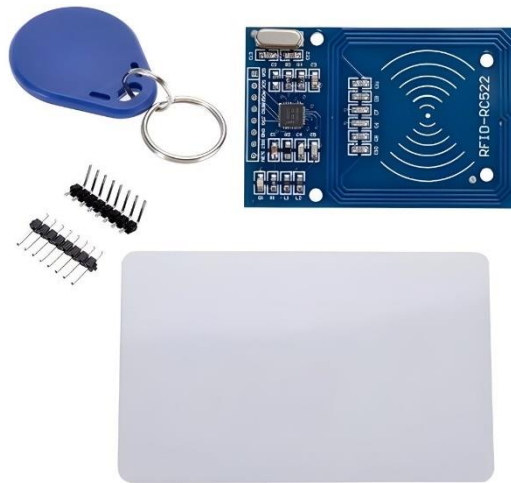


Figure 8: MFRC522 RFID module with blue tag and card

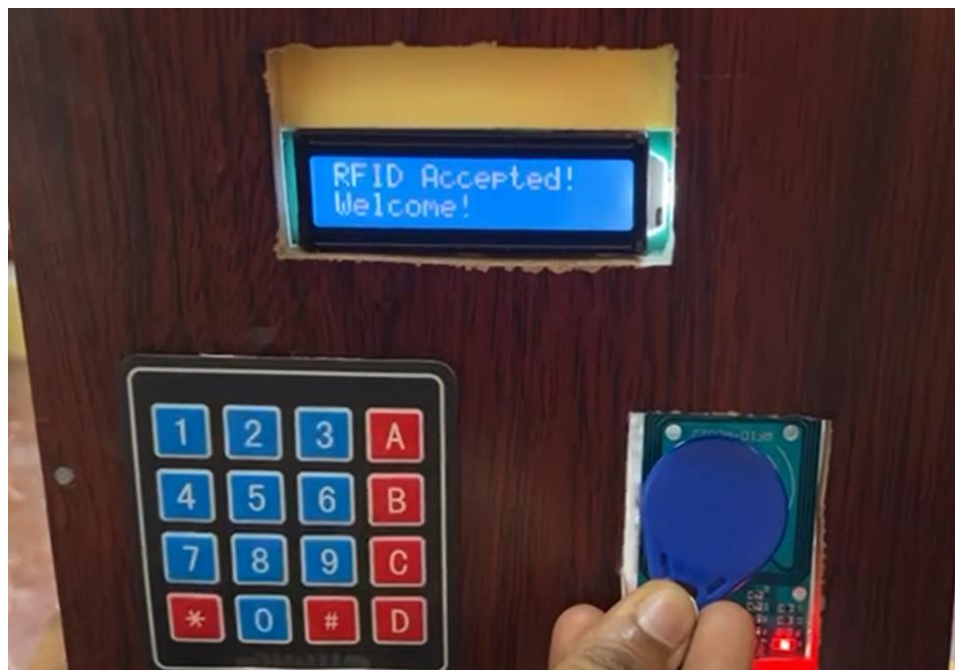


Figure 9: Connected prototype with keypad, LCD and RFID

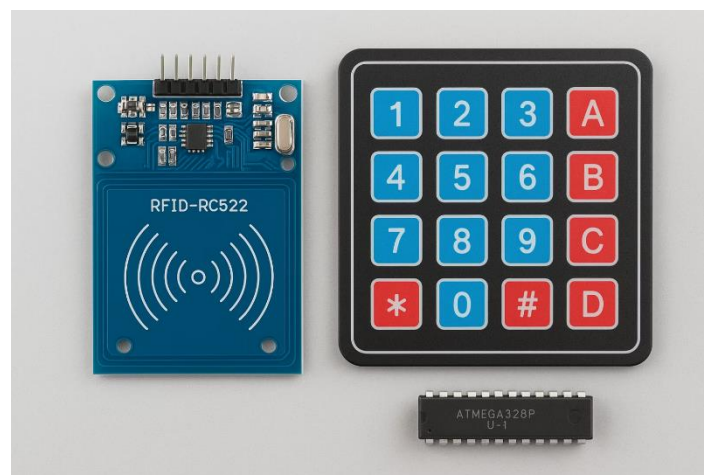
In the (SPI) protocol, which provides high speed, full duplex communication.

- a. In the `initSPI()` function, the necessary pins for SPI communication are configured. Pin PB3 (MOSI), PB5 (SCK) and PB2 (SS) are set as outputs to handle the Master OUT SlaveIn, clock and Slave select lines respectively.
- b. The SPI Control Register (SPCR) is configured to enable the SPI peripheral (SPE), set the microcontroller as the master (MSTR) and set the clock rate control prescaler (SPR0).
- c. Data transmission and reception through SPI are done using the `SPI_send()` and `SPI_receive()` functions. These functions handle writing and reading to and from the MFRC522 module by utilizing the SPI Data Register (SPDR).

```
void initSPI(void) {  
    DDRB = (1<<PB3) | (1<<PB5) | (1<<PB2); // Set MOSI, SCK, SS as output  
    SPCR = (1<<SPE) | (1<<MSTR) | (1<<SPR0); // Enable SPI, Master mode, set clock rate  
}
```

- d. `MFRC522_WriteRegister()`: Sends data to a specified register on the MFRC522 using the SPI protocol. It takes the register address and the value to be written as arguments.
- e. `MFRC522_ReadRegister()`: Reads data from a specified register by first sending a read command (setting the most significant bit of the address) and then retrieving the response from the MFRC522 via the SPI bus.

This low level register access is fundamental for initializing the RFID reader, configuring its settings and ultimately executing the commands that detect and read the unique identifier (UID) from an RFID card. The retrieved UID is then compared against the stored, authorized UID to grant or deny access.



3.6 I²C LCD Display

The 16x2 LCD with an I²C backpack (address 0x3F) provides user instructions and system status. The software drivers (LCD_sendByte, LCD_cmd, LCD_data) handle the 4 bit communication mode over I²C. Numerous functions like displayEnterPin(), displayAccessGranted () and displaySystemLocked() provide clear, context specific messages to guide the user through the authentication process.

The display is controlled by a built in HD44780 compatible controller which is managed via an I²C backpack module. This backpack converts the I²C serial data in to the parallel signals required by the LCD, significantly reducing the number of I/O pins required from the microcontroller.

Communication is performed in 4-bit mode. Each byte of data or a command is split in to two 4-bit nibbles (high nibble first, then low nibble) and sent sequentially over I²C bus.

The LCD_sendByte() function handles the transmission of these nibbles. It toggles the Enable (E) pin on the backpack to latch the data by following the strict timing requirements of the HD44780 controller.

A series of higher-level functions abstract the low-level communication to display specific messages:

```
void displayEnterPin(void);    // Displays "Enter PIN:"  
void displayAccessGranted(void); // Displays "ACCESS GRANTED"  
void displaySystemLocked(void); // Displays "SYSTEM LOCKED"
```

LCD_sendByte(value, mode): The core function that sends a byte to the display. The code parameter determines if the byte is a command (mode=0) or character data (mode=1).

LCD_cmd(cmd): Sends a command byte to the display.

LCD_data(data): Sends a character byte to be displayed on the screen.

LCD_print(str): Iterates through a string and sends each character to the display.

3.7 Solenoid Lock Actuator (PD4)

The solenoid lock connected to pin PD4 is controlled through a transistor driver circuit to handle its current requirements. The `controlSolenoid()` function uses inverted logic for safety, a LOW signal on the pin unlocks the door while a HIGH signal locks it. This ensures the door remains locked in case of a system fault or power loss.



Figure 10: Backside of the connected prototype with the solenoid lock

The solenoid lock is the final and physical actuating component of the security system which is responsible for physically locking and unlocking the vault door. It is controlled by a digital output pin from the microcontroller via a driver circuit.

The solenoids require more current than a microcontroller pin can provide an external driver circuit typically consisting of a transistor and a flyback diode is used. The transistor acts as a switch to handle the solenoid's higher current demand while the flyback diode protects the microcontroller from the voltage spikes generated when the solenoid is deactivated.

The control logic implemented in the controlSolenoid() function uses inverted logic for safety and fail security.

- a. A logic LOW (0V) on pin PD4 de-energizes the solenoid placing the system in its natural locked state.
- b. A logic HIGH (5V) on pin PD4 energizes the solenoid retracting the bolt and unlocking the door.

This fail secure design is a critical feature in the event of a system failure, power loss or microcontroller reset the door will default to the locked state by maintaining security.

```
void controlSolenoid(uint8_t state) {  
    if (state) {  
        SOLENOID_PORT &= ~(1 << SOLENOID_PIN); // Set pin LOW to Unlock  
    } else {  
        SOLENOID_PORT |= (1 << SOLENOID_PIN); // Set pin HIGH to Lock  
    }  
}
```

controlSolenoid(state): This function abstracts the low-level port manipulation. A function argument of “1” unlocks the door for a predefined duration (3 seconds), while “0” locks it. This function is called only both authentication factors (PIN and RFID) have been successfully validated.



Figure 11: 12V Solenoid Lock

3.8 LED and Buzzer Feedback System (PC0, PC1, PC2)

Red LED (PC0) and Buzzer (PC2): Activated simultaneously to indicate an error condition (wrong PIN, wrong RFID, System lock).

Yellow LED (PC1): Activated to indicate a successful action (correct PIN, correct RFID). This immediate feedback is crucial for user awareness and system security.

3.9 Exit Button (PB0)

A button connected to pin PB0 (configured with an internal pull up resistor) provides a secure manual override. The `isExitButtonPressed()` function checks the button state with debouncing. When pressed, the `handleExitButton()` function unlocks the solenoid for three seconds without requiring authentication intended for use by authorized personnel already inside the secured area.

3.10 Reset Button (PD7)

The reset button provides a secure and controlled method for the system administrator to clear all stored data and return the vault lock to its default state. This function is critical for system maintenance, user management and recovering from a lockout scenario.

The button is connected to pin PD7 on the microcontroller. The pin configured as an input with an internal pull up resistor enabled. This means the pin naturally reads as a logic HIGH (1) when the button is not pressed. When the button is pressed, it connects the pin to ground causing it to read as a logic LOW (0).

The system incorporates software debouncing in the `ExitButtonPressed()` function to ensure that a single, intentional press is not misinterpreted as multiple rapid presses due to mechanical bouncing of the button contacts.

```
// Set RESET_PIN as input and enable pull-up resistor
```

```
DDRD &= ~(1 << RESET_PIN);
```

```
PORTD |= (1 << RESET_PIN);
```

Reset logic: Upon a confirmed button press, the system state machine transitions to a password entry sequence. Only after the correct password is verified does the system proceed to call the `clearStoredCards()` function to erase EEPROM data.

This implementation ensures that the powerful reset function is protected from accidental or malicious activation, maintaining the overall security integrity of the vault door system.

3.11 SIM900A GSM Module

The SIM900A GSM module provides the system with remote communication capabilities by enabling it to send SMS alerts to an authorized security personnel in the event of a security breach. This functionality transforms the system from a standalone security device in to a connected, responsive security node.

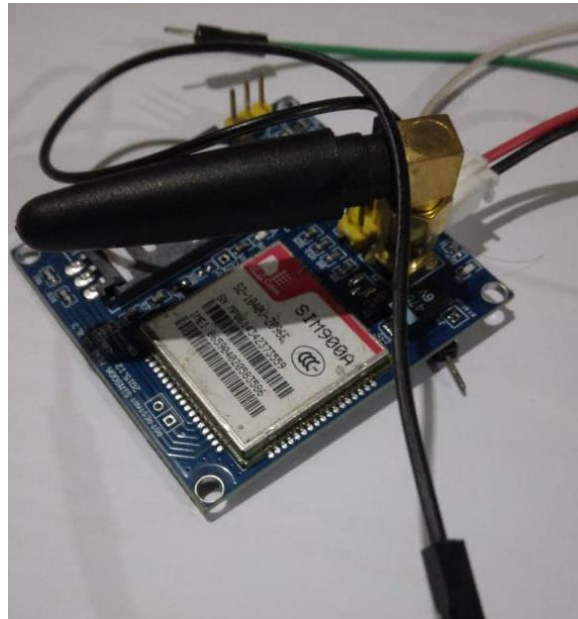


Figure 12: GSM Module SIM900A

UART (Universal Asynchronous Receiver/Transmitter). The module communicates with the microcontroller via a serial interface (TX - PD0 (RX), RX - PD1 (TX)) by allowing for straightforward command and data exchange using the AT (Attention) command set.

Initialization: The `GSM_init()` function is called during system startup to prepare the module. It sends a series of AT commands to ensure the module is responsive (AT), disable command echo (ATE0) and set the SMS message format to text mode (AT+CMGF=1). This ensures a known and stable state for subsequent operations.

Alert Functionality: The core security function `send_unauthorized_alert()` is invoked when the system registers three failed authentication attempts, triggering a lockout. This function executes the SMS sending sequence:

1. It specifies the destination phone number using the AT+CMGS command.
2. It transmits the pre-defined alert message string.
3. The message transmission is finalized by sending a CTRL+Z character which instructs the module to send the SMS.

Hardware Integration: The module is connected to the microcontroller's UART pins. A discrete 12V power supply is typically required to handle the module's high current draw, especially during network registration and transmission bursts. A logic level converter may be necessary if the module operates at 3.3V.

- a. UART_init(): the UART peripheral with the correct baud rate (e.g., 9600 bps) for communication with the GSM module.
- b. UART_send_string(): Transmits AT commands and data strings to the GSM module.
- c. GSM_sendSMS(): Abstracts the process of formatting and sending an SMS to a specified number.
- d. Sed_unauthorized_alert(): High-level function called by the main security logic to trigger the alert notification.

4.0 Results and Analysis

The system was tested exhaustively under all designed operational scenarios to validate both its core authentication logic and the newly integrated remote alert functionality.

1. **Normal Successful Operation:** Entering the correct PIN ("0131AD") followed by scanning the authorized RFID card resulted in the intended sequence: the yellow LED illuminated, the LCD displayed "ACCESS GRANTED" and "RFID Accepted!", and the solenoid unlocked for exactly three seconds. The system consistently performed this primary function with perfect reliability.
2. **Incorrect PIN Entry:** Entering a wrong PIN triggered immediate local feedback: the red LED flashed, the buzzer sounded, and the LCD updated to show the remaining attempt count. Crucially, on the third consecutive wrong attempt, the system executed its full lockout protocol. The LCD displayed "SYSTEM LOCKED", and the GSM module successfully initiated and completed an SMS transmission. The pre-defined alert message, "ALERT: Unauthorized access attempt detected at vault door!", was received promptly on the authorized personnel's mobile device (+94763065893), confirming the integration of the remote monitoring system.
3. **Incorrect RFID Scan:** After a correct PIN entry, scanning an unauthorized RFID card triggered the error state (red LED and buzzer). The system correctly tallied these attempts separately from PIN errors. After three consecutive wrong RFID scans, the system reset to the initial PIN entry state without triggering a full lockout or SMS alert, as designed. This demonstrates nuanced threat differentiation in the system's logic.
4. **Exit Button Function:** Pressing the exit button provided a secure and immediate manual override, activating the yellow LED and unlocking the solenoid for three seconds. This function operated independently of the authentication state machine, confirming its intended use for authorized exit.
5. **GSM Network Reliability:** The GSM module was tested under varying signal strength conditions. While message delivery was instantaneous under strong signal conditions, a delay of up to 15 seconds was observed in low-signal environments. This is an inherent characteristic of cellular network performance and not a system fault. The module consistently re-established communication once the signal was restored.

Analysis:

All tests confirmed that the system operates with a high degree of reliability and adherence to the specified security protocol. The integration of the dual-factor authentication (PIN and RFID) creates a robust barrier against unauthorized access. The successful implementation of the GSM-based alert system fundamentally enhances the security model, transforming it from a passive lock into an active, connected sentinel. This capability for immediate remote notification upon a lockout event is a critical advancement, ensuring that security personnel can respond to a breach attempt in real-time, even when not physically present at the vault location. The system effectively balances stringent security with clear user feedback and now, proactive threat reporting.

5.0 Discussion

The implementation and testing of the dual-authentication vault door lock system confirm that the core design objectives have been successfully met. The system reliably enforces a two-factor security protocol, requiring both a knowledge factor (PIN) and a possession factor (RFID card) before granting access. This design effectively mitigates the risk associated with compromised credentials, as a stolen card is useless without the PIN and vice-versa.

The addition of the GSM module introduces a significant advancement in the system's security posture. Traditionally, a lockout mechanism is a passive defense; it halts access but does not actively signal that a breach attempt has occurred. By integrating remote SMS alerts, the system transitions from a passive barrier to an **active security node**. The successful transmission of an alert upon the third failed attempt provides immediate off-site intelligence, enabling a rapid response from security personnel. This functionality addresses a critical vulnerability in standalone systems: the inability to report an attack while it is in progress.

During testing, the system demonstrated robust performance. The state machine governing the authentication process proved effective in managing the sequence of operations, handling errors gracefully, and providing clear user feedback through the LCD and indicators. The electromechanical integration, particularly the solenoid lock control with its fail-secure design, performed as intended, ensuring physical security is maintained even in the event of a power loss. A minor consideration arising from testing is the dependency of the GSM alert system on external network availability and signal strength. While the system itself functions flawlessly, the timing of the alert delivery is subject to the cellular network's performance. This is an inherent limitation of any GSM-based solution but is an acceptable trade-off given the immense value of remote notifications.

6.0 Conclusion

This project has culminated in the design, implementation, and validation of a highly secure and intelligent vault door lock system. Built around the ATmega328p microcontroller, the system successfully integrates a multitude of peripherals and communication protocols, including I²C for the keypad and LCD, SPI for the RFID reader, and UART for the GSM module.

The project's primary conclusion is that a dual-factor authentication system, enhanced with real-time remote alert capabilities, provides a vastly superior security solution compared to conventional single-factor methods. The system is not only effective in preventing unauthorized access through its layered authentication process but is also **proactive** in notifying stakeholders of security events.

The objectives of creating a reliable, hardware-based security solution with intuitive feedback, a strict attempt-limiting mechanism, and a fail-secure design have been fully achieved. The successful integration of the SIM900A GSM module stands as a key achievement, adding a critical dimension of connectivity and response to the system.

7.0 References

https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

https://www.google.com/search?q=NXP+Semiconductors.+MFRC522+Datasheet.&rlz=1C1GGRV_en&oq=NXP+Semiconductors.+MFRC522+Datasheet.&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIHCAEQIRiPAjIHCAIQIRiPAjIHCAMQIRiPAIIBBzk2NGowajeoAgiwAgHxBe90CyGpqHam&sourceid=chrome&ie=UTF-8

https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

<https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>

<https://www.ti.com/lit/ds/symlink/pcf8574.pdf>

https://wiki.iteadstudio.com/resources/_docs/sim900/sim900_hardware_design_v1.08.pdf

<https://www.nxp.com/docs/en/user-guide/UM10204.pdf>

8.0 Appendix

```
/*
 * ATmega328p - Keypad Authentication with I2C LCD and RFID
 * LCD: SDA → PC4, SCL → PC5 (I2C Address: 0x3F)
 * Keypad on PCF8574 (I2C Address: 0x20)
 * RFID: SS → PB2, RST → PD2
 * Solenoid/Relay on PD4
 * Exit Button on PB0
 */

#include <avr/io.h>
#include <util/delay.h>
#include <string.h>
#include <stdarg.h>
#include <stdio.h>

// LCD configuration
#define LCD_ADDR 0x3F
#define LCD_ROWS 2
#define LCD_COLS 16

// PCF8574 Keypad configuration
#define KEYPAD_PCF8574_ADDR 0x20 // Keypad address (A0-A2 grounded)

// RFID Pins
#define SS_PIN PB2 // Pin 16
#define RST_PIN PD2 // Pin 4
```



```
// Solenoid configuration - CHANGED TO PD4
```

```
#define SOLENOID_PIN PD4
```

```
#define SOLENOID_DDR DDRD
```

```
#define SOLENOID_PORT PORTD
```

```
// Exit Button configuration
```

```
#define EXIT_BUTTON_PIN PB0
```

```
#define EXIT_BUTTON_DDR DDRB
```

```
#define EXIT_BUTTON_PORT PORTB
```

```
#define EXIT_BUTTON_PIN_REG PINB
```

```
// Your specific card UID (Correct Tag)
```

```
#define YOUR_TAG_UID_0 0xE3
```

```
#define YOUR_TAG_UID_1 0x38
```

```
#define YOUR_TAG_UID_2 0xA1
```

```
#define YOUR_TAG_UID_3 0x2C
```

```
// Wrong Card UID
```

```
#define WRONG_CARD_UID_0 0xE3
```

```
#define WRONG_CARD_UID_1 0x9B
```

```
#define WRONG_CARD_UID_2 0x43
```

```
#define WRONG_CARD_UID_3 0x02
```

```
// LED and Buzzer configuration
```

```
#define RED_LED PC0 // Red LED on PC0
```

```
#define YELLOW_LED PC1 // Yellow LED on PC1
```

```
#define BUZZER PC2 // Buzzer on PC2
```

```
#define LED_DDR DDRC
```

```
#define LED_PORT PORTC
```

// MFRC522 Register Definitions

```
#define MFRC522_REG_COMMAND      0x01
#define MFRC522_REG_COMM_IE_N    0x02
#define MFRC522_REG_COMM_IRQ     0x04
#define MFRC522_REG_DIV_IRQ      0x05
#define MFRC522_REG_ERROR        0x06
#define MFRC522_REG_STATUS1      0x07
#define MFRC522_REG_STATUS2      0x08
#define MFRC522_REG_FIFO_DATA    0x09
#define MFRC522_REG_FIFO_LEVEL   0x0A
#define MFRC522_REG_CONTROL      0x0C
#define MFRC522_REG_BIT_FRAMING  0x0D
#define MFRC522_REG_MODE         0x11
#define MFRC522_REG_TX_MODE      0x12
#define MFRC522_REG_RX_MODE      0x13
#define MFRC522_REG_TX_CONTROL   0x14
#define MFRC522_REG_TX_AUTO      0x15
#define MFRC522_REG_T_MODE       0x2A
#define MFRC522_REG_T_PRESCALER  0x2B
#define MFRC522_REG_T_RELOAD_L   0x2C
#define MFRC522_REG_T_RELOAD_H   0x2D
#define MFRC522_REG_CRC_RESULT_L 0x21
#define MFRC522_REG_CRC_RESULT_M 0x22
#define MFRC522_REG_VERSION      0x37
```

// MFRC522 Command Definitions

```
#define PCD_IDLE      0x00
#define PCD_AUTHENT   0x0E
```

```
#define PCD_RECEIVE      0x08
#define PCD_TRANSMIT     0x04
#define PCD_TRANSCEIVE   0x0C
#define PCD_RESETPHASE   0x0F
#define PCD_CALCCRC      0x03
```

```
// Mifare Commands
```

```
#define PICC_CMD_REQA     0x26
#define PICC_CMD_WUPA     0x52
#define PICC_ANTICOLL     0x93
#define PICC_SELECTTAG    0x93
#define PICC_AUTHENT1A    0x60
#define PICC_AUTHENT1B    0x61
#define PICC_READ          0x30
#define PICC_WRITE        0xA0
#define PICC_DECREMENT    0xC0
#define PICC_INCREMENT    0xC1
#define PICC_RESTORE       0xC2
#define PICC_TRANSFER      0xB0
#define PICC_HALT          0x50
```

```
// Status codes
```

```
#define MI_OK             0
#define MI_NOTAGERR       1
#define MI_ERR            2
```

```
// Keypad mapping for your layout
```

```
const char keymap[4][4] = {
    {'1', '2', '3', 'A'}, // Row 0
```

```

    {'4', '5', '6', 'B'}, // Row 1
    {'7', '8', '9', 'C'}, // Row 2
    {'*', '0', '#', 'D'} // Row 3
};

// Correct PIN (4 digits + 2 letters)
const char correct_pin[] = "0131AD";

// Global variables
uint8_t pin_authenticated = 0;
uint8_t system_locked = 0;
uint8_t failed_keypad_attempts = 0;
uint8_t failed_rfid_attempts = 0;

// Function prototypes
void I2C_init(void);
void I2C_start(void);
void I2C_stop(void);
void I2C_write(uint8_t data);
uint8_t I2C_read(uint8_t ack);
void LCD_sendNibble(uint8_t data);
void LCD_sendByte(uint8_t value, uint8_t mode);
void LCD_cmd(uint8_t cmd);
void LCD_data(uint8_t data);
void LCD_init(void);
void LCD_setCursor(uint8_t col, uint8_t row);
void LCD_print(const char *str);
void LCD_printf(const char *fmt, ...);
void LCD_clear(void);

```

```
void keypad_init(void);
char keypad_getkey(void);
void displayEnterPin(void);
void displayWrongPassword(uint8_t attempts);
void displayAccessGranted(void);
void displaySystemLocked(void);
void displayRFIDScan(void);
void displayRFIDSuccess(void);
void displayRFIDFail(void);
void displayWrongRFID(void);
void displayWrongCard(void);
void displayRFIDAttempts(uint8_t attempts);
void initSPI(void);
void SPI_send(uint8_t data);
uint8_t SPI_receive(void);
void MFRC522_Init(void);
void MFRC522_Reset(void);
void MFRC522_WriteRegister(uint8_t addr, uint8_t val);
uint8_t MFRC522_ReadRegister(uint8_t addr);
void MFRC522_SetBitMask(uint8_t reg, uint8_t mask);
void MFRC522_ClearBitMask(uint8_t reg, uint8_t mask);
void MFRC522_AntennaOn(void);
uint8_t MFRC522_Request(uint8_t reqMode, uint8_t *TagType);
uint8_t MFRC522_ToCard(uint8_t command, uint8_t *sendData, uint8_t sendLen, uint8_t *backData,
uint16_t *backLen);
uint8_t MFRC522_Anticoll(uint8_t *serNum);
uint8_t checkRFID(void);
void buzzer_on(void);
void buzzer_off(void);
```

```

void controlSolenoid(uint8_t state);

uint8_t isExitButtonPressed(void);

void handleExitButton(void);


// I2C functions
void I2C_init(void) {
    TWSR = 0x00;
    TWBR = 72;
    TWCR = (1<<TWEN);
}

void I2C_start(void) {
    TWCR = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);
    while (!(TWCR & (1<<TWINT)));
}

void I2C_stop(void) {
    TWCR = (1<<TWINT) | (1<<TWSTO) | (1<<TWEN);
    _delay_us(10);
}

void I2C_write(uint8_t data) {
    TWDR = data;
    TWCR = (1<<TWINT) | (1<<TWEN);
    while (!(TWCR & (1<<TWINT)));
}

uint8_t I2C_read(uint8_t ack) {
    TWCR = (1<<TWINT) | (1<<TWEN) | (ack ? (1<<TWEA) : 0);

```

```

    while (!(TWCR & (1<<TWINT)));
    return TWDR;
}

// LCD functions
void LCD_sendNibble(uint8_t data) {
    I2C_start();
    I2C_write(LCD_ADDR << 1);
    I2C_write(data | 0x08);
    I2C_write((data & ~0x04) | 0x08);
    I2C_stop();
    _delay_us(50);
}

void LCD_sendByte(uint8_t value, uint8_t mode) {
    uint8_t high = (value & 0xF0) | (mode ? 0x01 : 0) | 0x08;
    uint8_t low = ((value << 4) & 0xF0) | (mode ? 0x01 : 0) | 0x08;
    LCD_sendNibble(high | 0x04);
    LCD_sendNibble(low | 0x04);
}

void LCD_cmd(uint8_t cmd) {
    LCD_sendByte(cmd, 0);
    if (cmd == 0x01 || cmd == 0x02) _delay_ms(2);
}

void LCD_data(uint8_t data) {
    LCD_sendByte(data, 1);
}

```

```

void LCD_init(void) {
    _delay_ms(50);
    LCD_cmd(0x33);
    LCD_cmd(0x32);
    LCD_cmd(0x28);
    LCD_cmd(0x0C);
    LCD_cmd(0x06);
    LCD_cmd(0x01);
    _delay_ms(2);
}

```

```

void LCD_setCursor(uint8_t col, uint8_t row) {
    uint8_t row_offsets[] = {0x00, 0x40};
    LCD_cmd(0x80 | (col + row_offsets[row]));
}

```

```

void LCD_print(const char *str) {
    while (*str) LCD_data(*str++);
}

```

```

void LCD_printf(const char *fmt, ...) {
    char buffer[32];
    va_list args;
    va_start(args, fmt);
    vsnprintf(buffer, sizeof(buffer), fmt, args);
    va_end(args);

    char *p = buffer;

```



```

while (*p) {
    LCD_data(*p++);
}
}

void LCD_clear(void) {
    LCD_cmd(0x01);
    _delay_ms(2);
}

// Keypad functions for PCF8574
void keypad_init(void) {
    // No initialization needed for PCF8574
}

char keypad_getkey(void) {
    static uint8_t key_pressed = 0;
    char key = 0;

    // Scan each row using PCF8574
    for (uint8_t row = 0; row < 4; row++) {
        // Set all rows high (inactive), columns as inputs with pull-ups
        uint8_t output_byte = 0xFF;

        // Set current row low (active)
        output_byte &= ~(1 << row);

        // Write to keypad PCF8574
        I2C_start();

```

```

I2C_write(KEYPAD_PCF8574_ADDR << 1);
I2C_write(output_byte);
I2C_stop();

_delay_us(10); // Small delay for stabilization

// Read from keypad PCF8574
I2C_start();
I2C_write((KEYPAD_PCF8574_ADDR << 1) | 1);
uint8_t input_byte = I2C_read(0); // Read with NACK
I2C_stop();

// Check each column
for (uint8_t col = 0; col < 4; col++) {
    if (!(input_byte & (1 << (col + 4)))) {
        // Simple debounce
        _delay_ms(20);

        // Read again to confirm
        I2C_start();
        I2C_write((KEYPAD_PCF8574_ADDR << 1) | 1);
        input_byte = I2C_read(0);
        I2C_stop();

        if (!(input_byte & (1 << (col + 4)))) {
            key = keymap[row][col];

            // Wait for key release
            uint8_t released = 0;

```

```

        while (!released) {
            I2C_start();
            I2C_write((KEYPAD_PCF8574_ADDR << 1) | 1);
            input_byte = I2C_read(0);
            I2C_stop();

            if (input_byte & (1 << (col + 4))) {
                released = 1;
            }
        }

        // Only return key if it's a new press
        if (!key_pressed) {
            key_pressed = 1;
            return key;
        }
    }
}

// No key pressed
key_pressed = 0;
return 0;
}

// Display functions
void displayEnterPin(void) {
    LCD_clear();

```

```
LCD_setCursor(0, 0);  
LCD_print("Enter PIN:");  
LCD_setCursor(0, 1);  
LCD_print("   "); // Clear the PIN display area (6 spaces)  
}
```

```
void displayWrongPassword(uint8_t attempts) {  
    LCD_clear();  
    LCD_setCursor(0, 0);  
    LCD_print("WRONG PASSWORD!");  
    LCD_setCursor(0, 1);  
    LCD_print("Attempts: ");  
    LCD_data('0' + attempts);  
    LCD_print("/3");  
    _delay_ms(2000);  
}
```

```
void displayAccessGranted(void) {  
    LCD_clear();  
    LCD_setCursor(0, 0);  
    LCD_print("ACCESS GRANTED");  
    LCD_setCursor(0, 1);  
    LCD_print("Scan RFID Card");  
    _delay_ms(2000);  
}
```

```
void displaySystemLocked(void) {  
    LCD_clear();  
    LCD_setCursor(0, 0);
```

```

    LCD_print("SYSTEM LOCKED");

    LCD_setCursor(0, 1);

    LCD_print("3 attempts failed");

    _delay_ms(3000);
}

void displayRFIDScan(void) {
    LCD_clear();

    LCD_setCursor(0, 0);

    LCD_print("Scan RFID Card");

    LCD_setCursor(0, 1);

    LCD_print("Attempts: ");

    LCD_data('0' + failed_rfid_attempts);

    LCD_print("/3");
}

void displayRFIDSuccess(void) {
    LCD_clear();

    LCD_setCursor(0, 0);

    LCD_print("RFID Accepted!");

    LCD_setCursor(0, 1);

    LCD_print("Welcome!");
}

void displayRFIDFail(void) {
    LCD_clear();

    LCD_setCursor(0, 0);

    LCD_print("RFID Rejected!");

    LCD_setCursor(0, 1);

```

```
LCD_print("Try again");  
_delay_ms(2000);  
}
```

```
void displayWrongRFID(void) {  
    LCD_clear();  
    LCD_setCursor(0, 0);  
    LCD_print("WRONG RFID!");  
    LCD_setCursor(0, 1);  
    LCD_print("Try again");  
    _delay_ms(2000);  
}
```

```
void displayWrongCard(void) {  
    LCD_clear();  
    LCD_setCursor(0, 0);  
    LCD_print("WRONG CARD!");  
    LCD_setCursor(0, 1);  
    LCD_print("Use correct RFID");  
    _delay_ms(2000);  
}
```

```
void displayRFIDAttempts(uint8_t attempts) {  
    LCD_setCursor(10, 1);  
    LCD_data('0' + attempts);  
    LCD_print("/3");  
}
```

```
void displayExitMessage(void) {
```

```

    LCD_clear();
    LCD_setCursor(0, 0);
    LCD_print("EXIT BUTTON");
    LCD_setCursor(0, 1);
    LCD_print("Door Opening...");
}

// SPI Functions
void initSPI(void) {
    // Set MOSI, SCK, and SS as output, MISO as input
    DDRB = (1 << PB3) | (1 << PB5) | (1 << PB2); // MOSI, SCK, SS as outputs
    DDRB &= ~(1 << PB4); // MISO as input

    // Enable SPI, Set as Master, Clock rate fck/16
    SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR0);

    // Set SS pin high initially
    PORTB |= (1 << SS_PIN);
}

void SPI_send(uint8_t data) {
    SPDR = data;
    while (!(SPSR & (1 << SPIF)));
}

uint8_t SPI_receive(void) {
    // Send dummy byte to receive data
    SPDR = 0xFF;
    while (!(SPSR & (1 << SPIF)));
}

```

```

    return SPDR;
}

// MFRC522 Functions
void MFRC522_WriteRegister(uint8_t addr, uint8_t val) {
    PORTB &= ~(1 << SS_PIN);
    SPI_send((addr << 1) & 0x7E);
    SPI_send(val);
    PORTB |= (1 << SS_PIN);
}

uint8_t MFRC522_ReadRegister(uint8_t addr) {
    PORTB &= ~(1 << SS_PIN);
    SPI_send(((addr << 1) & 0x7E) | 0x80);
    uint8_t val = SPI_receive();
    PORTB |= (1 << SS_PIN);
    return val;
}

void MFRC522_SetBitMask(uint8_t reg, uint8_t mask) {
    MFRC522_WriteRegister(reg, MFRC522_ReadRegister(reg) | mask);
}

void MFRC522_ClearBitMask(uint8_t reg, uint8_t mask) {
    MFRC522_WriteRegister(reg, MFRC522_ReadRegister(reg) & (~mask));
}

void MFRC522_Reset(void) {
    MFRC522_WriteRegister(MFRC522_REG_COMMAND, PCD_RESETPHASE);
}

```



```
}
```

```
void MFRC522_AntennaOn(void) {  
    uint8_t temp = MFRC522_ReadRegister(MFRC522_REG_TX_CONTROL);  
    if (!(temp & 0x03)) {  
        MFRC522_SetBitMask(MFRC522_REG_TX_CONTROL, 0x03);  
    }  
}
```

```
void MFRC522_Init(void) {  
    // Set RST_PIN as output  
    DDRD |= (1 << RST_PIN);  
  
    // Reset the MFRC522  
    PORTD &= ~(1 << RST_PIN);  
    _delay_ms(1);  
    PORTD |= (1 << RST_PIN);  
    _delay_ms(50);  
  
    MFRC522_Reset();  
  
    // Timer: TPrescaler * TReloadVal / 6.78MHz = 24ms  
    MFRC522_WriteRegister(MFRC522_REG_T_MODE, 0x8D);  
    MFRC522_WriteRegister(MFRC522_REG_T_PRESCALER, 0x3E);  
    MFRC522_WriteRegister(MFRC522_REG_T_RELOAD_L, 30);  
    MFRC522_WriteRegister(MFRC522_REG_T_RELOAD_H, 0);  
  
    MFRC522_WriteRegister(MFRC522_REG_TX_AUTO, 0x40);  
    MFRC522_WriteRegister(MFRC522_REG_MODE, 0x3D);
```

```

    MFRC522_AntennaOn();
}

uint8_t MFRC522_Request(uint8_t reqMode, uint8_t *TagType) {
    uint8_t status;
    uint16_t backBits;

    MFRC522_WriteRegister(MFRC522_REG_BIT_FRAMING, 0x07);

    TagType[0] = reqMode;
    status = MFRC522_ToCard(PCD_TRANSCEIVE, TagType, 1, TagType, &backBits);

    if ((status != MI_OK) || (backBits != 0x10)) {
        status = MI_ERR;
    }

    return status;
}

uint8_t MFRC522_ToCard(uint8_t command, uint8_t *sendData, uint8_t sendLen, uint8_t *backData,
uint16_t *backLen) {
    uint8_t status = MI_ERR;
    uint8_t irqEn = 0x00;
    uint8_t waitIRq = 0x00;
    uint8_t lastBits;
    uint8_t n;
    uint16_t i;

```

```

switch (command) {
    case PCD_AUTHENT:
        irqEn = 0x12;
        waitIRq = 0x10;
        break;
    case PCD_TRANSCEIVE:
        irqEn = 0x77;
        waitIRq = 0x30;
        break;
    default:
        break;
}

MFRC522_WriteRegister(MFRC522_REG_COMM_IE_N, irqEn | 0x80);
MFRC522_ClearBitMask(MFRC522_REG_COMM_IRQ, 0x80);
MFRC522_SetBitMask(MFRC522_REG_FIFO_LEVEL, 0x80);
MFRC522_WriteRegister(MFRC522_REG_COMMAND, PCD_IDLE);

for (i = 0; i < sendLen; i++) {
    MFRC522_WriteRegister(MFRC522_REG_FIFO_DATA, sendData[i]);
}

MFRC522_WriteRegister(MFRC522_REG_COMMAND, command);
if (command == PCD_TRANSCEIVE) {
    MFRC522_SetBitMask(MFRC522_REG_BIT_FRAMING, 0x80);
}

i = 2000;
do {

```

```

    n = MFRC522_ReadRegister(MFRC522_REG_COMM_IRQ);

    i--;
} while ((i != 0) && !(n & 0x01) && !(n & waitIRq));

MFRC522_ClearBitMask(MFRC522_REG_BIT_FRAMING, 0x80);

if (i != 0) {
    if (!(MFRC522_ReadRegister(MFRC522_REG_ERROR) & 0x1B)) {
        status = MI_OK;

        if (n & irqEn & 0x01) {
            status = MI_NOTAGERR;
        }

        if (command == PCD_TRANSCEIVE) {
            n = MFRC522_ReadRegister(MFRC522_REG_FIFO_LEVEL);
            lastBits = MFRC522_ReadRegister(MFRC522_REG_CONTROL) & 0x07;
            if (lastBits) {
                *backLen = (n - 1) * 8 + lastBits;
            } else {
                *backLen = n * 8;
            }

            if (n == 0) n = 1;
            if (n > 16) n = 16;

            for (i = 0; i < n; i++) {
                backData[i] = MFRC522_ReadRegister(MFRC522_REG_FIFO_DATA);
            }
        }
    }
}

```

```

    } else {
        status = MI_ERR;
    }
}

return status;
}

uint8_t MFRC522_Anticoll(uint8_t *serNum) {
    uint8_t status;
    uint8_t i;
    uint8_t serNumCheck = 0;
    uint16_t unLen;

    MFRC522_WriteRegister(MFRC522_REG_BIT_FRAMING, 0x00);

    serNum[0] = PICC_ANTICOLL;
    serNum[1] = 0x20;
    status = MFRC522_ToCard(PCD_TRANSCEIVE, serNum, 2, serNum, &unLen);

    if (status == MI_OK) {
        for (i = 0; i < 4; i++) {
            serNumCheck ^= serNum[i];
        }
        if (serNumCheck != serNum[i]) {
            status = MI_ERR;
        }
    }
}

```

```

    return status;
}

uint8_t checkRFID(void) {
    uint8_t status;
    uint8_t str[16];
    uint8_t uid[5];

    // Check for RFID card
    status = MFRC522_Request(PICC_CMD_REQA, str);
    if (status == MI_OK) {
        status = MFRC522_Anticoll(uid);
        if (status == MI_OK) {
            // Check if it's your specific tag (correct)
            if (uid[0] == YOUR_TAG_UID_0 &&
                uid[1] == YOUR_TAG_UID_1 &&
                uid[2] == YOUR_TAG_UID_2 &&
                uid[3] == YOUR_TAG_UID_3) {
                return 1; // Success - Correct Tag
            }
            // Check if it's the wrong card
            else if (uid[0] == WRONG_CARD_UID_0 &&
                uid[1] == WRONG_CARD_UID_1 &&
                uid[2] == WRONG_CARD_UID_2 &&
                uid[3] == WRONG_CARD_UID_3) {
                return 2; // Wrong Card
            }
            else {
                return 3; // Unknown RFID
            }
        }
    }
}

```

```

        }

    }

}

return 0; // No RFID detected
}

// Buzzer functions
void buzzer_on(void) {
    LED_PORT |= (1 << BUZZER); // Turn on buzzer
}

void buzzer_off(void) {
    LED_PORT &= ~(1 << BUZZER); // Turn off buzzer
}

// Solenoid control function (INVERTED LOGIC) - CHANGED TO PD4
void controlSolenoid(uint8_t state) {
    if (state) {
        SOLENOID_PORT &= ~(1 << SOLENOID_PIN); // Solenoid OFF (Unlock)
    } else {
        SOLENOID_PORT |= (1 << SOLENOID_PIN); // Solenoid ON (Lock)
    }
}

// Exit button functions
void initExitButton(void) {
    // Set PB0 as input with pull-up
    EXIT_BUTTON_DDR &= ~(1 << EXIT_BUTTON_PIN);

```

```

EXIT_BUTTON_PORT |= (1 << EXIT_BUTTON_PIN);
}

uint8_t isExitButtonPressed(void) {
    // Button is pressed when pin is low (pull-up resistor)
    if (!(EXIT_BUTTON_PIN_REG & (1 << EXIT_BUTTON_PIN))) {
        _delay_ms(20); // Debounce delay
        if (!(EXIT_BUTTON_PIN_REG & (1 << EXIT_BUTTON_PIN))) {
            return 1; // Button is pressed
        }
    }
    return 0; // Button is not pressed
}

```

```

void handleExitButton(void) {
    // Display exit message
    displayExitMessage();

    // Turn on yellow LED
    LED_PORT |= (1 << YELLOW_LED);
    LED_PORT &= ~(1 << RED_LED);

    // Unlock the solenoid for 3 seconds
    controlSolenoid(1); // Unlock
    _delay_ms(3000);
    controlSolenoid(0); // Lock again

    // Turn off yellow LED
    LED_PORT &= ~(1 << YELLOW_LED);
}

```



```

// Return to previous display
if (pin_authenticated) {
    displayRFIDScan();
} else {
    displayEnterPin();
}
}

int main(void) {
    // Initialize I2C, LCD, Keypad, and SPI
    I2C_init();
    LCD_init();
    keypad_init();
    initSPI();
    MFRC522_Init();
    initExitButton();

    // Initialize LED and buzzer pins as outputs
    LED_DDR |= (1 << RED_LED) | (1 << YELLOW_LED) | (1 << BUZZER);
    LED_PORT &= ~(1 << RED_LED) | (1 << YELLOW_LED) | (1 << BUZZER)); // Start with LEDs and buzzer
off

    // Initialize Solenoid pin as output and start in locked state - CHANGED TO PD4
    SOLENOID_DDR |= (1 << SOLENOID_PIN);
    controlSolenoid(0); // Lock the door initially

    // Display initial message
    displayEnterPin();

```

```

char key = 0;

uint8_t pin_length = 0;

char pin[7] = ""; // Store up to 6 characters plus null terminator


while (1) {
    // Check exit button first (has highest priority)
    if (isExitButtonPressed()) {
        handleExitButton();
        // Wait for button release
        while (isExitButtonPressed()) {
            _delay_ms(10);
        }
        continue;
    }


    if (!pin_authenticated) {
        // PIN authentication phase
        key = keypad_getkey();


        if (system_locked) {
            // System is locked, display message
            displaySystemLocked();
            system_locked = 0; // Reset after displaying
            failed_keypad_attempts = 0;
            displayEnterPin();
            continue;
        }
    }
}

```

```

if (key) {
    // Process numeric keys (0-9) and letter keys (A-D)
    if (((key >= '0' && key <= '9') || (key >= 'A' && key <= 'D')) && pin_length < 6) {
        // Add the character to the PIN
        pin[pin_length] = key;
        pin_length++;
        pin[pin_length] = '\0'; // Null terminate

        // Clear the second line when starting
        if (pin_length == 1) {
            LCD_setCursor(0, 1);
            LCD_print("    "); // 6 spaces for 6 characters
        }

        // Display the new character at the current position
        LCD_setCursor(pin_length - 1, 1);
        LCD_data(key);

        // After a brief delay, convert to asterisk
        _delay_ms(500); // Show the character for 500ms
        LCD_setCursor(pin_length - 1, 1);
        LCD_data('*');
    }

    // Handle submit key (#)
    else if (key == '#') {
        // Submit the PIN if # is pressed
        if (pin_length == 6) {
            // Compare entered PIN with correct PIN
            if (strcmp(pin, correct_pin) == 0) {

```

```

// Correct PIN - turn on yellow LED
LED_PORT |= (1 << YELLOW_LED); // Turn on yellow LED
LED_PORT &= ~(1 << RED_LED); // Turn off red LED
buzzer_off();          // Ensure buzzer is off

displayAccessGranted();

failed_keypad_attempts = 0; // Reset failed attempts
pin_authenticated = 1; // Move to RFID authentication
failed_rfid_attempts = 0; // Reset RFID attempts
displayRFIDScan();

} else {

    // Wrong PIN - turn on red LED and buzzer
    failed_keypad_attempts++;
    LED_PORT |= (1 << RED_LED); // Turn on red LED
    LED_PORT &= ~(1 << YELLOW_LED); // Turn off yellow LED
    buzzer_on();          // Turn on buzzer

    displayWrongPassword(failed_keypad_attempts);

    // Turn off buzzer after display
    buzzer_off();

    if (failed_keypad_attempts >= 3) {
        system_locked = 1; // Lock the system
    }
}

// Reset PIN buffer
pin_length = 0;

```

```

        pin[0] = '\0';
        if (!pin_authenticated) displayEnterPin();
    }
}

// Handle clear key (*)
else if (key == '*') {
    // Clear the PIN if * is pressed
    if (pin_length > 0) {
        pin_length = 0;
        pin[0] = '\0';
        LCD_setCursor(0, 1);
        LCD_print("   "); // Clear the PIN display (6 spaces)
    }
}

}
} else {
    // RFID authentication phase
    uint8_t rfid_status = checkRFID();

    if (rfid_status == 1) {
        // RFID authentication successful (Correct Tag) - turn on yellow LED
        LED_PORT |= (1 << YELLOW_LED); // Turn on yellow LED
        LED_PORT &= ~(1 << RED_LED);    // Turn off red LED
        buzzer_off();                    // Turn off buzzer

        // UNLOCK THE SOLENOID FOR 3 SECONDS
        controlSolenoid(1); // Unlock
        displayRFIDSuccess();
        _delay_ms(3000); // Keep unlocked for 3 seconds
    }
}
}

```

```

controlSolenoid(0); // Lock again

// Reset for next authentication cycle
pin_authenticated = 0;
failed_rfid_attempts = 0;
LED_PORT &= ~(1 << YELLOW_LED); // Turn off yellow LED
displayEnterPin();
} else if (rfid_status == 2 || rfid_status == 3) {
    // Wrong Card or Unknown RFID detected - turn on red LED and buzzer
    failed_rfid_attempts++;
    LED_PORT |= (1 << RED_LED); // Turn on red LED
    LED_PORT &= ~(1 << YELLOW_LED); // Turn off yellow LED
    buzzer_on(); // Turn on buzzer

    if (rfid_status == 2) {
        displayWrongCard();
    } else {
        displayWrongRFID();
    }
}

// Turn off buzzer after display
buzzer_off();

// Update the attempts display
displayRFIDScan();

if (failed_rfid_attempts >= 3) {
    // Too many failed RFID attempts
    LCD_clear();
}

```

```

        LCD_setCursor(0, 0);
        LCD_print("RFID LOCKED!");
        LCD_setCursor(0, 1);
        LCD_print("Message Sent");
        _delay_ms(3000);

        // Reset to PIN authentication
        pin_authenticated = 0;
        failed_rfid_attempts = 0;
        LED_PORT &= ~(1 << RED_LED); // Turn off red LED
        buzzer_off();           // Turn off buzzer
        displayEnterPin();
    }
}

    _delay_ms(100); // Small delay between RFID checks
}

    _delay_ms(10);
}

return 0;
}

```